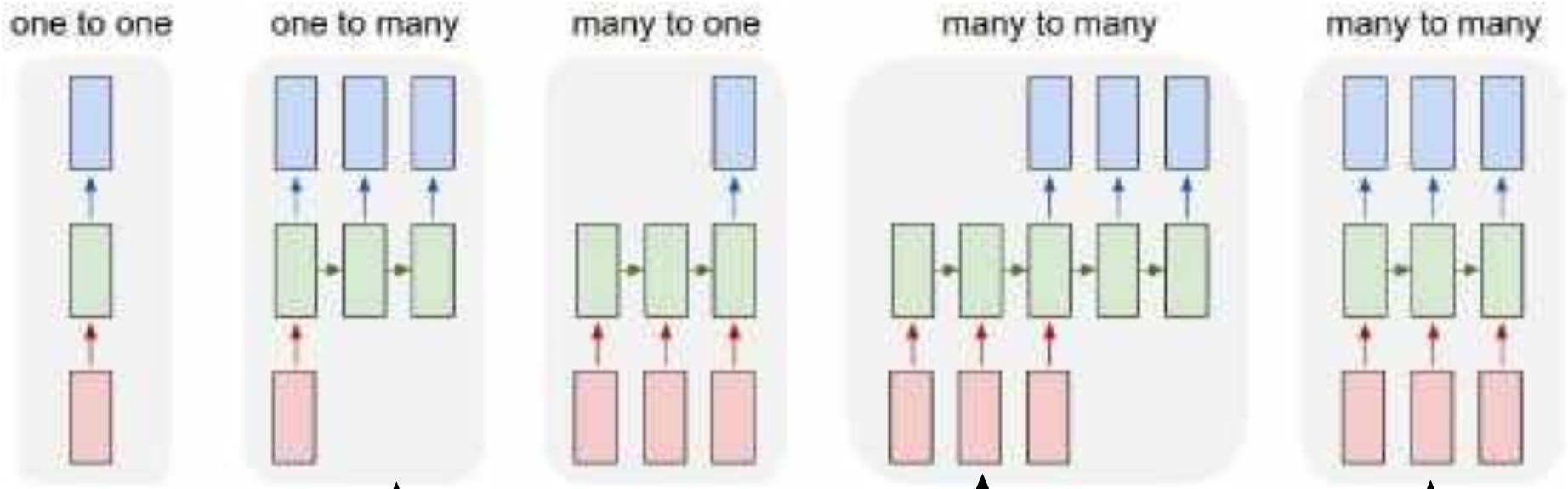
Recurrent Neural Networks (RNN)

- Standard NN models (MLPs, CNNs) are not able to handle sequences of data
  - They accept a fixed-sized vector as input and produce a fixed-sized vector as output
  - The weights are updated independent of the order the samples are processed
- RNNs are designed for modeling **sequences**
  - Sequences in the input, in the output or in both
  - They are capable of remembering past information

Recurrent Neural Networks (RNN)



e.g., **Image Captioning**
image-> sequence of words

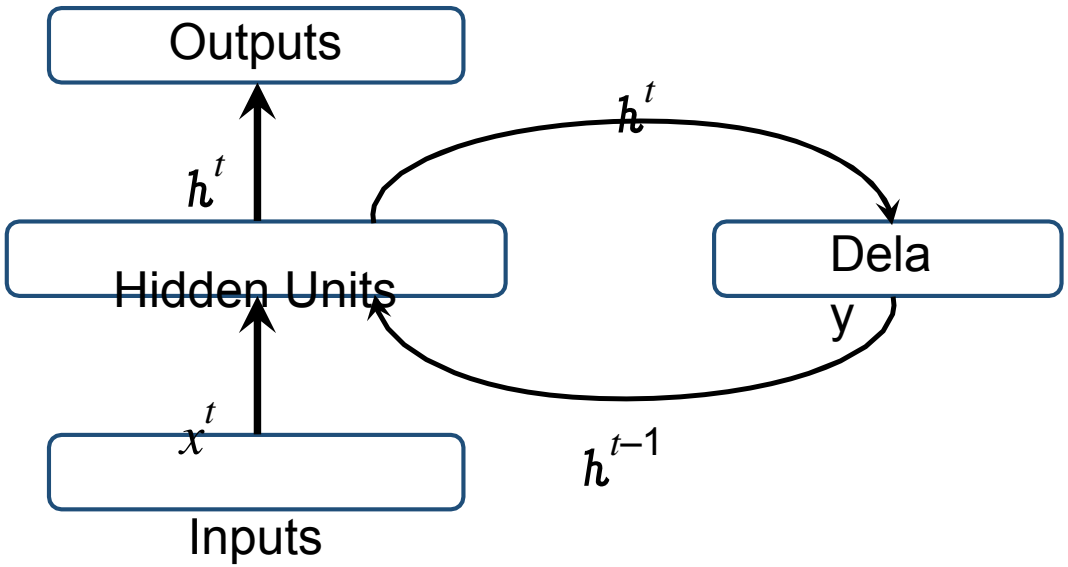
e.g., **Sentiment Classification**
sequence of words -> sentiment

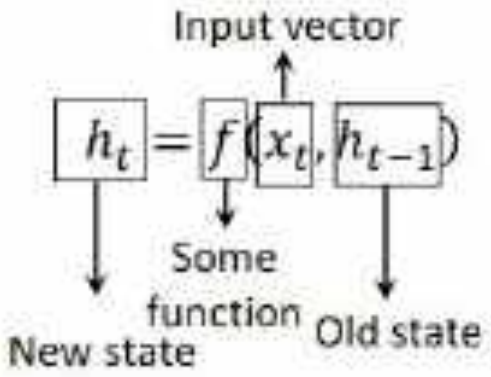
e.g., **Machine Translation**
sequence of words -> sequence of words

e.g., **Frame Level Video Classification**
sequence of frames -> sequence of labels

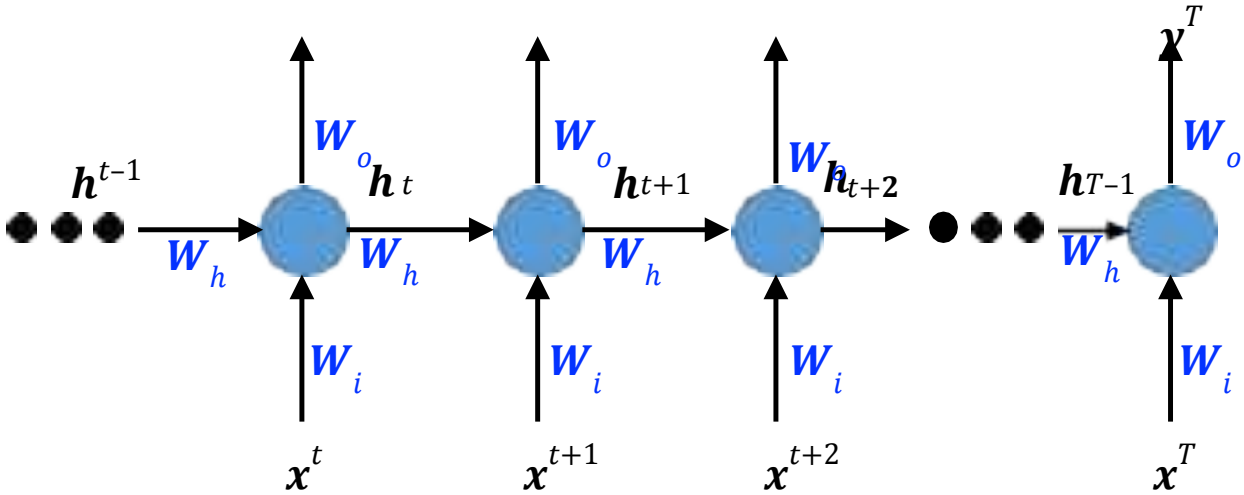
Recurrent Neural Networks (RNN)

- The fundamental feature of a Recurrent Neural Network (RNN) is that the network contains at least one feedback connection so that activation can flow in a loop.
- The feedback connection allows information to persist. Remember the generation of people would require the generation of several to be remembered.
- The simplest form of RNN has the previous set of hidden unit activations feeding back into the network along with the inputs.





Recurrent Neural Networks (RNN) – Forward Pass



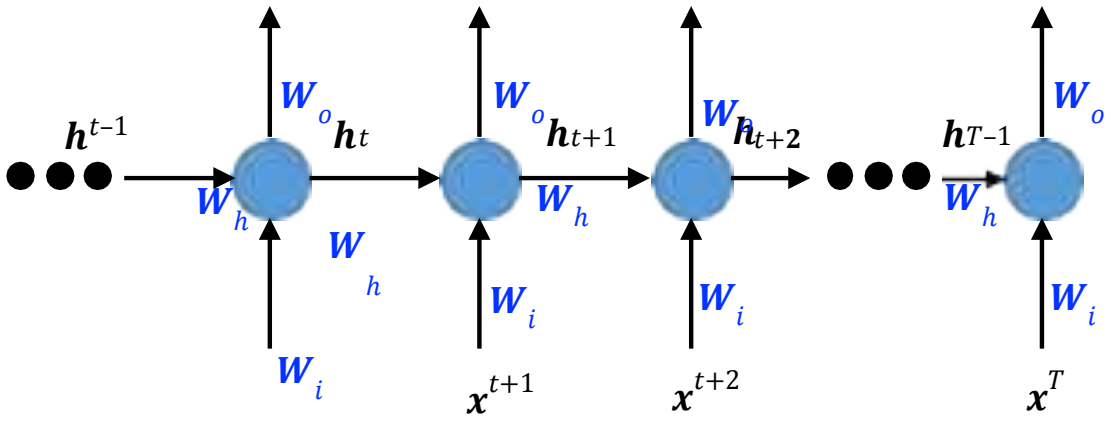
$$\mathbf{a}^t = \mathbf{W}_h\mathbf{h}^{t-1} + \mathbf{W}_i\mathbf{x}^t \quad (1)$$

$$\mathbf{h}^t = \mathbf{g}(\mathbf{a}^t) \quad (2)$$

$$\mathbf{y}^t = \mathbf{W}_o\mathbf{h}^t \quad (3)$$

§ Note that we can have biases too. For simplicity these are omitted.

Recurrent Neural Networks (RNN) - BPTT



$$\mathbf{a}^t = \mathbf{W}\,\mathbf{h}^{t-1} + \mathbf{W}_i\mathbf{x}^t$$
$$\mathbf{h}^t = \mathbf{g}(\mathbf{a}^t)$$
$$\mathbf{y}^t = \mathbf{W}_o\mathbf{h}^t$$

§ BPTT: Backpropagation through time

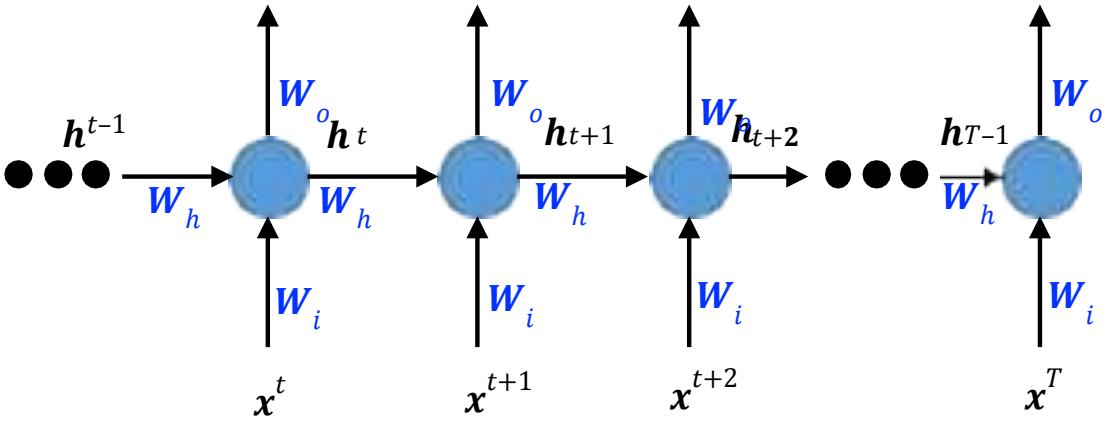
§ Total loss $L = \sum_{t=1}^{T} {}^tL$ and we are after $\partial L / \partial \mathbf{W}_o$, $\partial L / \partial \mathbf{W}_h$ and $\partial L / \partial \mathbf{W}_i$

§ Lets compute $\frac{\partial L}{\partial \mathbf{y}^t}$.

$$\frac{\partial L}{\partial \mathbf{y}^t} = \frac{\partial L}{\partial L^t}\frac{\partial L^t}{\partial \mathbf{y}^t} = 1.\frac{\partial L^t}{\partial \mathbf{y}^t} \qquad \boxed{} \qquad (4)$$

§ $\frac{\partial L_t}{\partial \mathbf{y}^t}$ is computable depending on the particular form of the loss function.

Recurrent Neural Networks (RNN) - BPTT



$$\mathbf{a}^t = \mathbf{W}_h \mathbf{h}^{t-1} + \mathbf{W}_i \mathbf{x}^t$$
$$\mathbf{h}^t = \mathbf{g}(\mathbf{a}^t)$$
$$\mathbf{y}^t = \mathbf{W}_o \mathbf{h}^t$$

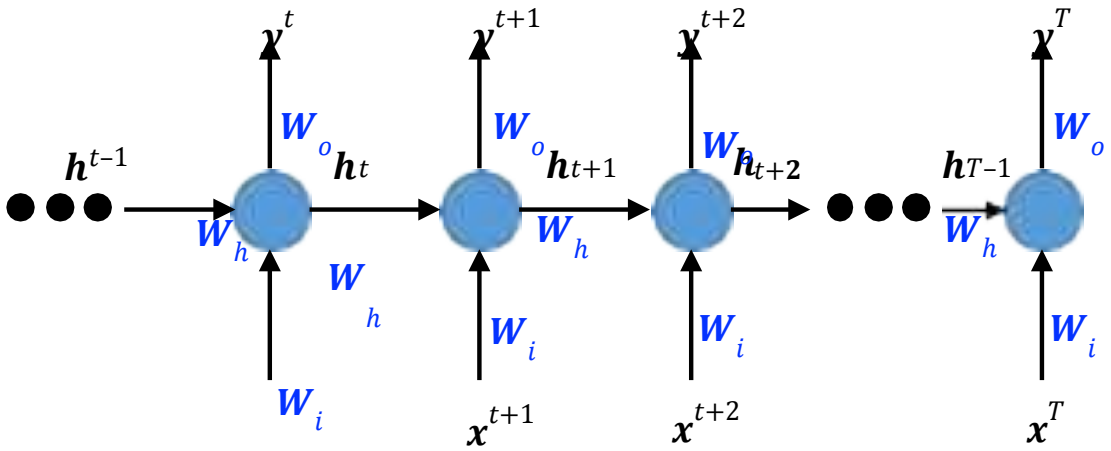
§ Lets compute $\frac{\partial L}{\partial \mathbf{h}^t}$. The subtlety here is that all L^t after timestep t are functions of $\mathbf{h}^t$. So, let us first consider $\frac{\partial L}{\partial \mathbf{h}^T}$, where T is the last timestep.

$$\frac{\partial L}{\partial \mathbf{h}^T} = \frac{\partial L}{\partial \mathbf{y}^T}\frac{\partial \mathbf{y}^T}{\partial \mathbf{h}^T} = \frac{\partial L}{\partial \mathbf{y}^T}\mathbf{W}_o \tag{5}$$

§ $\frac{\partial L}{\partial \mathbf{y}^T}$, we just computed last slide (eqn. (4))

§ For a generic t, we need to compute $\frac{\partial L}{\partial \mathbf{h}^t}$. $\mathbf{h}^t$ affects $\mathbf{y}^t$ and also $\mathbf{h}^{t+1}$. For this we will use something that we used while studying Backpropagation for feedforward networks.

Recurrent Neural Networks (RNN) - BPTT



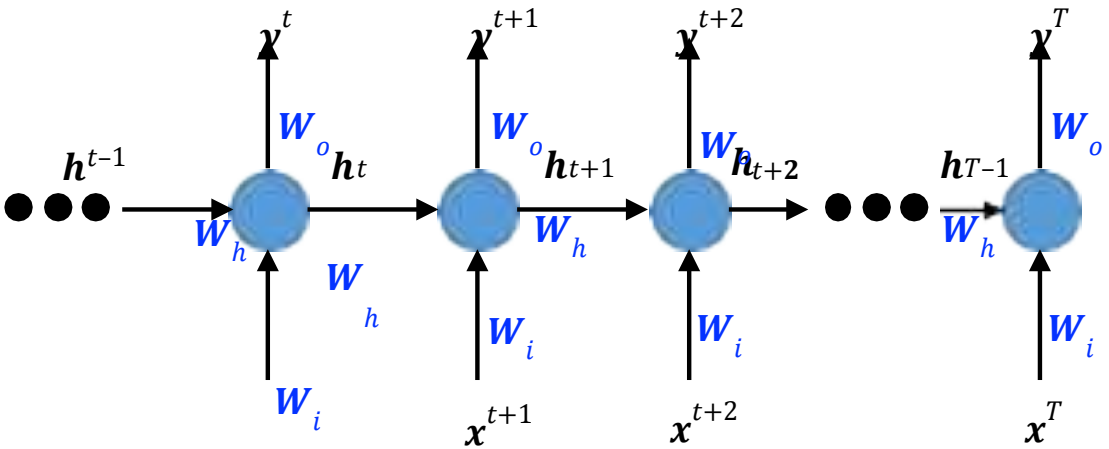
$$\mathbf{a}^t = \mathbf{W}_h \mathbf{h}^{t-1} + \mathbf{W}_i \mathbf{x}^t$$
$$\mathbf{h}^t = \mathbf{g}(\mathbf{a}^t)$$
$$\mathbf{y}^t = \mathbf{W}_o \mathbf{h}^t$$

§ If $u = f(x, y)$, where $x = \varphi(t)$, $y = \psi(t)$, then $\frac{\partial u}{\partial t} = \frac{\partial u}{\partial x}\frac{\partial x}{\partial t} + \frac{\partial u}{\partial y}\frac{\partial y}{\partial t}$

$$\frac{\partial L}{\partial \mathbf{h}^t} = \frac{\partial L}{\partial \mathbf{y}^t}\frac{\partial \mathbf{y}^t}{\partial \mathbf{h}^t} + \frac{\partial L}{\partial \mathbf{h}^{t+1}}\frac{\partial \mathbf{h}^{t+1}}{\partial \mathbf{h}^t} \qquad (6)$$

§ $\frac{\partial L}{\partial \mathbf{y}^t}$ we computed in eqn. (4)

Recurrent Neural Networks (RNN) - BPTT



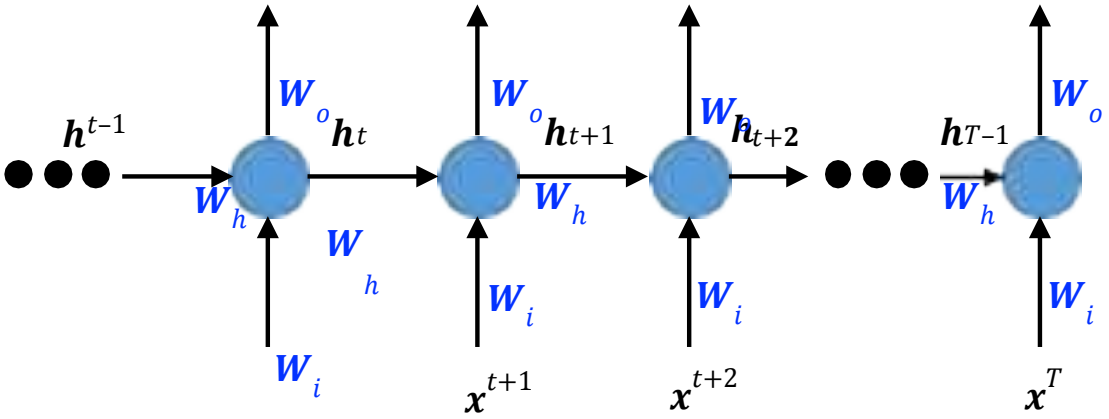
$$\mathbf{a}^t = \mathbf{W}_h \mathbf{h}^{t-1} + \mathbf{W}_i \mathbf{x}^t$$
$$\mathbf{h}^t = \mathbf{g}(\mathbf{a}^t)$$
$$\mathbf{y}^t = \mathbf{W}_o \mathbf{h}^t$$

§ If $u = f(x, y)$, where $x = \varphi(t)$, $y = \psi(t)$, then $\frac{\partial u}{\partial t} = \frac{\partial u}{\partial x}\frac{\partial x}{\partial t} + \frac{\partial u}{\partial y}\frac{\partial y}{\partial t}$

$$\frac{\partial L}{\partial \mathbf{h}^t} = \frac{\partial L}{\partial \mathbf{y}^t}\frac{\partial \mathbf{y}^t}{\partial \mathbf{h}^t} + \frac{\partial L}{\partial \mathbf{h}^{t+1}}\frac{\partial \mathbf{h}^{t+1}}{\partial \mathbf{h}^t} \qquad (6)$$

§ $\frac{\partial \mathbf{y}^t}{\partial \mathbf{h}^t} = \mathbf{W}_o$.

Recurrent Neural Networks (RNN) - BPTT



$$\mathbf{a}^t = \mathbf{W}\,\mathbf{h}^{t-1} + \mathbf{W}_i\mathbf{x}^t$$
$$\mathbf{h}^t = \mathbf{g}(\mathbf{a}^t)$$
$$\mathbf{y}^t = \mathbf{W}_o\mathbf{h}^t$$

§ If $u = f(x, y)$, where $x = \varphi(t)$, $y = \psi(t)$, then $\frac{\partial u}{\partial t} = \frac{\partial u}{\partial x}\frac{\partial x}{\partial t} + \frac{\partial u}{\partial y}\frac{\partial y}{\partial t}$

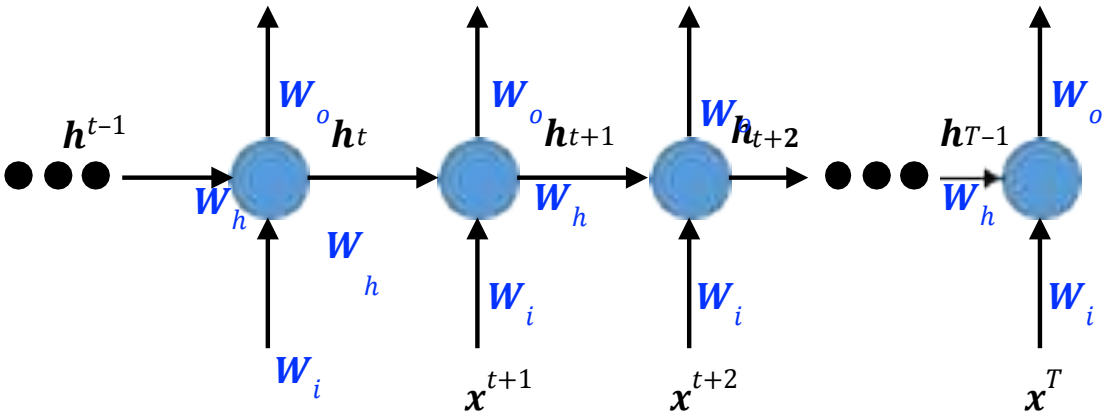
$$\frac{\partial L}{\partial \mathbf{h}^t} = \frac{\partial L}{\partial \mathbf{y}^t}\frac{\partial \mathbf{y}^t}{\partial \mathbf{h}^t} + \frac{\partial L}{\partial \mathbf{h}^{t+1}}\frac{\partial \mathbf{h}^{t+1}}{\partial \mathbf{h}^t} \qquad (6)$$

§ $\frac{\partial \mathbf{h}^{t+1}}{\partial \mathbf{h}^t} = \frac{\partial \mathbf{h}^{t+1}}{\partial \mathbf{a}^{t+1}}\frac{\partial \mathbf{a}^{t+1}}{\partial \mathbf{h}^t} = \mathbf{g}'\cdot\mathbf{W}$.

§ Since, $\mathbf{g}$ is an elementwise operation, $\mathbf{g}'$ will be a diagonal matrix.

§ In particular, if g is *tanh*, then $\frac{\partial \mathbf{h}^{t+1}}{\partial \mathbf{h}^t} = diag\left(1-(h_1^{t+1})^2, 1-(h_2^{t+1})^2, \ldots, 1-(h_m^{t+1})^2\right)$

Recurrent Neural Networks (RNN) - BPTT



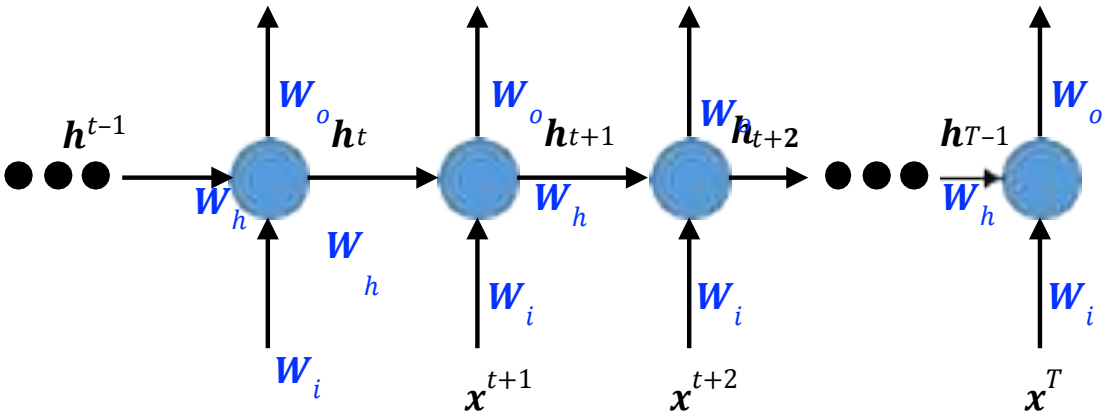
$$\mathbf{a}^t = \mathbf{W}_h \mathbf{h}^{t-1} + \mathbf{W}_i \mathbf{x}^t$$
$$\mathbf{h}^t = \mathbf{g}(\mathbf{a}^t)$$
$$\mathbf{y}^t = \mathbf{W}_o \mathbf{h}^t$$

- $\frac{\partial L}{\partial \mathbf{h}^t} = \frac{\partial L}{\partial \mathbf{y}^t}\frac{\partial \mathbf{y}^t}{\partial \mathbf{h}^t} + \frac{\partial L}{\partial \mathbf{h}^{t+1}}\frac{\partial \mathbf{h}^{t+1}}{\partial \mathbf{h}^t} = \frac{\partial L}{\partial \mathbf{y}^t}\mathbf{W}_o + \frac{\partial L}{\partial \mathbf{h}^{t+1}}\mathbf{diag}\left(1-(h_1^{t+1})^2, 1-(h_2^{t+1})^2, \ldots, 1-(h_m^{t+1})^2\right)\mathbf{W}_h$
- All the other things we can compute, but to compute $\frac{\partial L}{\partial \mathbf{h}^t}$ we need $\frac{\partial L}{\partial \mathbf{h}^{t+1}}$.
- From eqn. (5), we get $\frac{\partial L}{\partial \mathbf{h}^T}$, which gives $\frac{\partial L}{\partial \mathbf{h}^{T-1}}$ and so on.
- Now, $\frac{\partial L}{\partial \mathbf{W}_o} = \sum_t \frac{\partial L^t}{\partial \mathbf{W}_o} = \sum_t \frac{\partial L^t}{\partial \mathbf{y}_t}\frac{\partial \mathbf{y}_t}{\partial \mathbf{W}_o} = \sum_t \boxed{\frac{\partial L^t}{\partial \mathbf{y}^t}} \mathbf{h}^t$
- $\frac{\partial L_t}{\partial \mathbf{y}^t}$ is computable depending on the form of the loss function.
- (Do it yourself) Similarly for $\frac{\partial L}{\partial \mathbf{W}_h}$ and $\frac{\partial L}{\partial \mathbf{W}_i}$.

Exploding or Vanishing Gradients

- In recurrent nets (also in very deep nets), the final output is the composition of a large number of non-linear transformations.
- The derivatives through these compositions will tend to be either very small or very large.
- If $h = (f \circ g)(x) = f(g(x))$, then $h^r(x) = f^r(g(x))g^r(x)$
- If the gradients are small, the product is small.
- If the gradients are large, the product is large.

Exploding or Vanishing Gradients



$$\mathbf{a}^t = \mathbf{W}_h \mathbf{h}^{t-1} + \mathbf{W}_i \mathbf{x}^t$$
$$\mathbf{h}^t = \mathbf{g}(\mathbf{a}^t)$$
$$\mathbf{y}^t = \mathbf{W}_o \mathbf{h}^t$$

§ Let us see what happens with one learnable weight matrix $\boldsymbol{\theta} = \mathbf{W}_h$

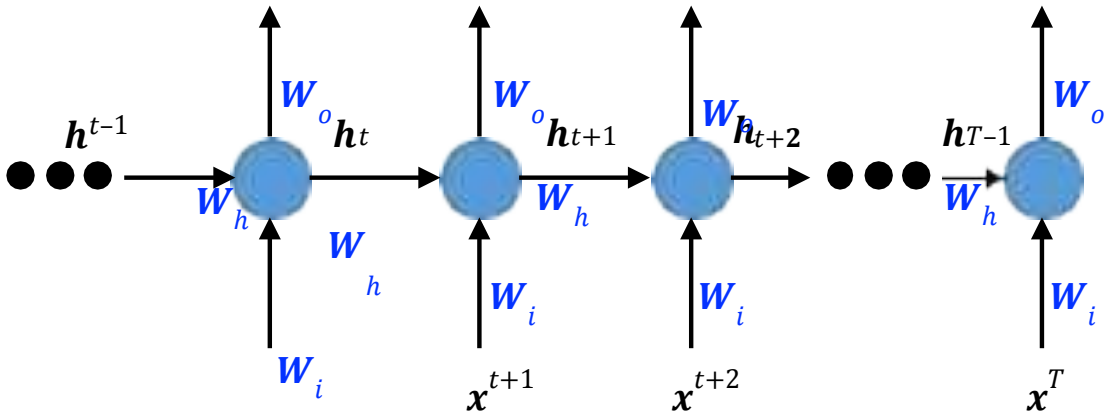
$$\frac{\partial L}{\partial \boldsymbol{\theta}} = \sum_{t=1}^{T} \frac{\partial L^t}{\partial \boldsymbol{\theta}}$$

$$\frac{\partial L^t}{\partial \boldsymbol{\theta}} = \frac{\partial L^t}{\partial \mathbf{y}^t} \frac{\partial \mathbf{y}^t}{\partial \boldsymbol{\theta}}$$

$$\frac{\partial \mathbf{y}^t}{\partial \boldsymbol{\theta}} = \frac{\partial \mathbf{y}^t}{\partial \mathbf{h}^t} \frac{\partial \mathbf{h}^t}{\partial \boldsymbol{\theta}}$$

§ But, $\mathbf{h}^t$ is a function of $\mathbf{h}^{t-1}, \mathbf{h}^{t-2}, \cdots, \mathbf{h}^2, \mathbf{h}^1$ and each of these is a function of $\boldsymbol{\theta}$.

Exploding or Vanishing Gradients



$$\mathbf{a}^t = \mathbf{W}_h \mathbf{h}^{t-1} + \mathbf{W}_i \mathbf{x}^t$$
$$\mathbf{h}^t = \mathbf{g}(\mathbf{a}^t)$$
$$\mathbf{y}^t = \mathbf{W}_o \mathbf{h}^t$$

§ Now we will resort to our friend again - If $u = f(x, y)$, where $x = \varphi(t)$, $y = \psi(t)$, then

$$\frac{\partial u}{\partial t} = \frac{\partial u}{\partial x}\frac{\partial x}{\partial t} + \frac{\partial u}{\partial y}\frac{\partial y}{\partial t}$$

$$\frac{\partial \mathbf{h}^t}{\partial \boldsymbol{\theta}} = \sum_{k=1}^{t} \frac{\partial \mathbf{h}^t}{\partial \mathbf{h}^k}\frac{\partial \mathbf{h}^k}{\partial \boldsymbol{\theta}}$$

§ And

$$\frac{\partial \mathbf{h}^t}{\partial \mathbf{h}^k} = \frac{\partial \mathbf{h}^t}{\partial \mathbf{h}^{t-1}}\frac{\partial \mathbf{h}^{t-1}}{\partial \mathbf{h}^{t-2}} \cdots \frac{\partial \mathbf{h}^{k+1}}{\partial \mathbf{h}^k}$$

$$= \mathrm{diag}\left[1 - (h_1^{t-1})^2, 1 - (h_2^{t-1})^2, \ldots\right] \mathrm{diag}\left[1 - (h_1^{t-2})^2, 1 - (h_2^{t-2})^2, \ldots\right] \cdots$$

Exploding or Vanishing Gradients

- Exploding Gradients
  - Easy to detect
  - Clip the gradient at a threshold
- Vanishing Gradients
  - More difficult to detect
  - Architectures designed to combat the problem of vanishing gradients.

Hopfield Networks

- A Hopfield net is composed of binary threshold units with recurrent connections between them. Recurrent networks of non-linear units are generally very hard to analyze. They can behave in many different ways:
  - Settle to a stable state
  - Oscillate
  - Follow chaotic trajectories that cannot be predicted far into the future.
- But Hopfield realized that if the connections are symmetric, there is a global energy function
  - Each “configuration” of the network has an energy.
  - The binary threshold decision rule causes the network to settle to an energy minimum.

Energy Function

- The global energy is the sum of many contributions. Each contribution depends on one connection weight and the binary states of two neurons:

$$E = -\sum_i s_i b_i \; - \sum_{i<j} s_i s_j w_{ij}$$

where bi is the bias of the ith unit, s is 0 or 1 depending on whether the unit is turned off or on respectively. And wij is the weight of the connection between units i and j.

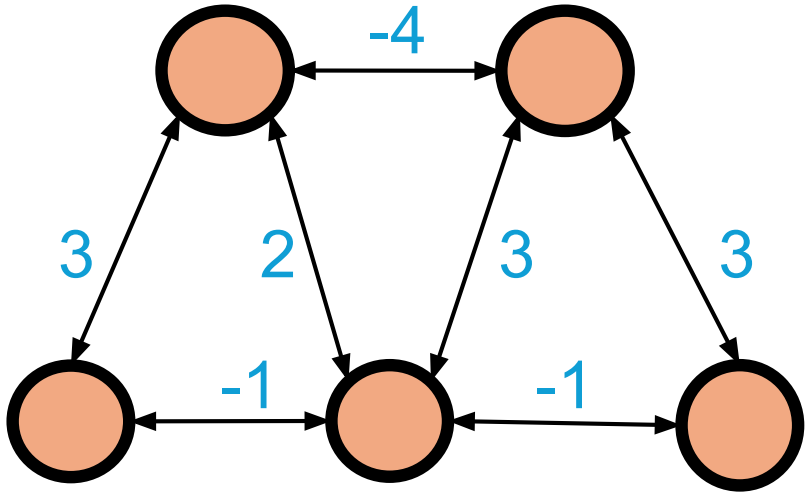
- The simple quadratic energy function makes it easy to compute how the state of one neuron affects the global energy:

$$E(s_i = 0) - E(s_i = 1) \quad = \quad b_i + \sum_j s_j w_{ij}$$

Settling to an energy minimum

The net is initially in a random state i.e., the units are on or off randomly. The binary threshold decision rule updates units one at a time in a random order.

1. Update each unit to whichever of its two states minimizes the global energy.
2. Use binary threshold units, i.e., the states can be either 0 or 1.



How to make use of this type of computation

- Hopfield proposed that memories could be energy minima of a neural net.
- The binary threshold decision rule can then be used to "clean up" incomplete or corrupted memories.
  - This gives a content-addressable memory in which an item can be accessed by just knowing part of its content (like google)
  - It is robust against hardware damage.

Storing memories

- If we use activities of 1 and -1, we can store a state vector by incrementing the weight between any two units by the product of their activities.
  - Treat biases as weights from a permanently on unit

$$\Delta w_{ij} = s_i s_j$$

- With states of 0 and 1 the rule is slightly more complicated.

$$\Delta w_{ij} = 4\,(s_i - \tfrac{1}{2})(s_j - \tfrac{1}{2})$$

Spurious Minima

- Each time we memorize a configuration, we hope to create a new energy minimum.

  But what if two nearby minima merge to create a minimum at an intermediate location?

  This limits the capacity of a Hopfield net.

- Using Hopfield's storage rule the capacity of a totally connected net with N units is only 0.15N memories.

  This does not make efficient use of the bits required to store the weights in the network.

Noisy networks find better energy minima

- A Hopfield net always makes decisions that reduce the energy.
  - This makes it impossible to escape from local minima.
- We can use random noise to escape from poor minima.
  - Start with a lot of noise so its easy to cross energy barriers.
  - Slowly reduce the noise so that the system ends up in a deep minimum.

Boltzmann Machines

- Boltzmann Machine was invented by Geoffrey Hinton and Terry Sejnowski in 1985.
- Is a type of stochastic RNN and extends Hopfield Network to two layered Network.
- Stochastic -its activation have a probabilistic element.
- Stochastic learning processes having recurrent structure and are the basis of the early optimization techniques used in ANN.
- To solve difficult combinatorial problems.

Structure of Boltzmann Machine

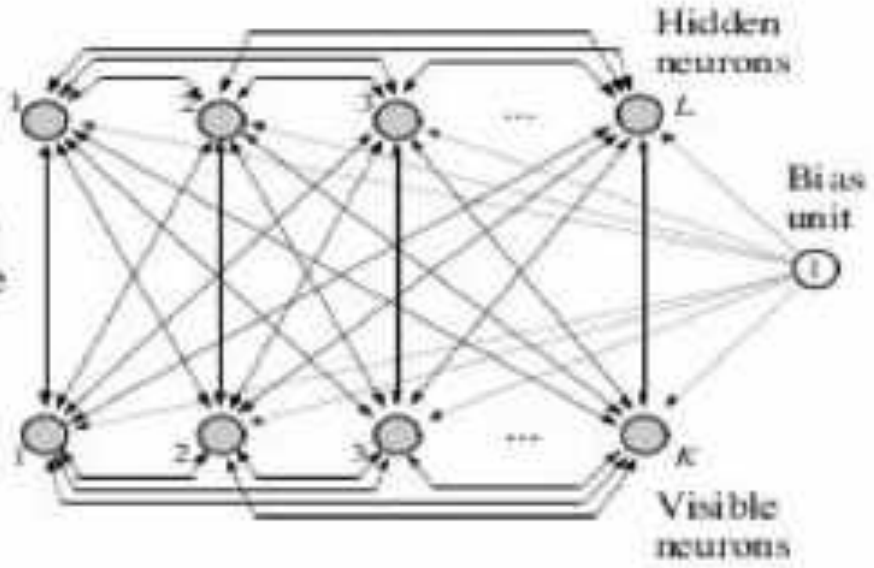


Figure 11.4: Architectural graph of Boltzmann machine; K is the number of visible neurons and L is the number of hidden neurons.

- The stochastic neurons of Boltzmann machine are in two groups: **vissible** and **hidden**.
- visible neurons provide an interface between the net and its environment.

Structure of Boltzmann Machines

- During the training phase, the visible neurons are clamped; the hidden neurons always operate freely, they are used to explain underlying constraints in the environmental input vectors.
- The hidden units do this (explain underlying constraints) by capturing higher-order correlations between the clamping vectors.

Structure of Boltzmann Machines

- Boltzmann machine learning may be viewed as an unsupervised learning procedure for modelling a distribution that is specified by the clamping patterns.
- the network can perform pattern completion: when a vector bearing part of the information is clamped onto a subset of the visible neurons, the network performs completion of the pattern on the remaining visible neurons (if it has learnt properly).

Objective of Boltzmann Machine

Modelling Binary Data: Given a training set of binary vectors, fit the model that will assign a probability to every possible binary vector.

When unit i is given opportunity to update its state, it first computes its total input z_i

When unit i is given opportunity to update its state, it first computes its total input z_i,

$$z_i = b_i + \sum_j s_j w_{ij} \tag{4}$$

Unit i turns on with probability -

$$prob(s_i = 1) = \frac{1}{1 + e^{-z_i}} \tag{5}$$

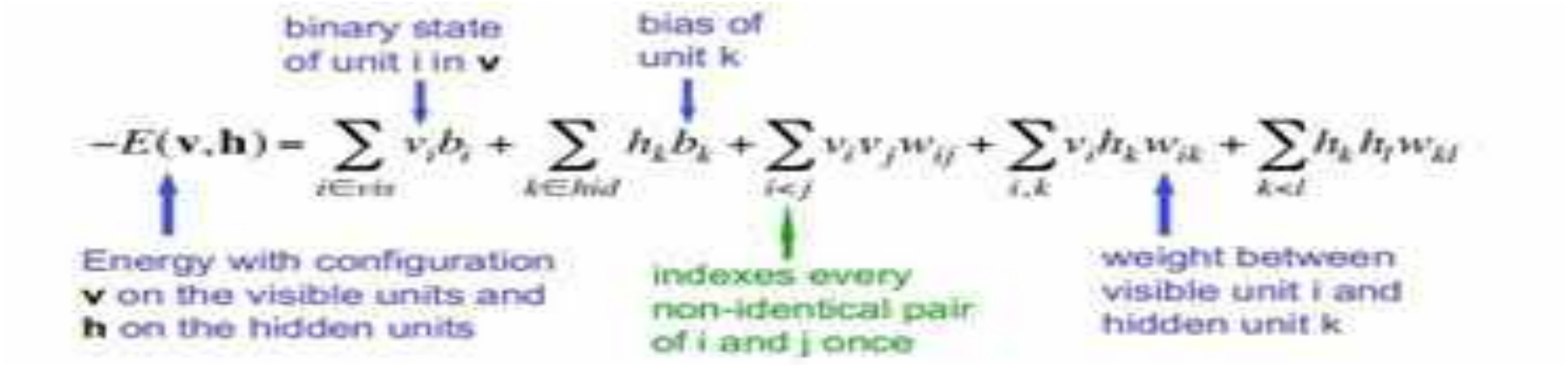
If the units are updated sequentially in random order, the network will eventually reach a Boltzmann Distribution, also called its stationary distribution.

How Boltzmann Machine Generates Data

- It is not a causal generative model.
- Everything is defined in terms of energies of joint configurations of the visible and hidden units.
- The energies of the joint configurations are related to their probabilities by two ways:

- either by defining the probability $p(\mathbf{v}, \mathbf{h}) \propto e^{-E(\mathbf{v},\mathbf{h})}$

- define the probability to be the probability of finding the network in that joint configuration after we have updated all of the stochastic binary units many times.



Probabilities in term of Energies

The probability of a joint configuration over both visible and hidden units depends on the energy of that joint configuration compared with the energies of all other joint configurations.

$$p(\mathbf{v},\mathbf{h}) = \frac{e^{-E(\mathbf{v},\mathbf{h})}}{\sum_{\mathbf{u},\mathbf{g}} e^{-E(\mathbf{u},\mathbf{g})}}$$

partition function

The probability of a configuration of the visible units is the sum of the probabilities of all the joint configurations that contain it.

$$p(\mathbf{v}) = \frac{\sum_{\mathbf{h}} e^{-E(\mathbf{v},\mathbf{h})}}{\sum_{\mathbf{u},\mathbf{g}} e^{-E(\mathbf{u},\mathbf{g})}}$$

Limitation and Solution

What if the network is remarkably large?

- If there are large number of hidden units then we cannot calculate partition function, as it is exponentially many terms.
- We need to sample the data and to do this we can use Markov Chain Monte Carlo(MCMC).
  - starting from a random global configuration pick units at random and allow them to stochastically update their states based on their energy gaps.
- Run MCMC until it reaches it stationary distribution(thermal eqb. at temp is 1)
  - the probability is relate

$$p(\mathbf{v}, \mathbf{h}) \propto e^{-E(\mathbf{v},\mathbf{h})}$$

Boltzmann Learning

Learning Algorithm for Boltzmann Machine is an unsupervised learning algorithm. Unlike Backpropagation Algorithm, where the training set consists of input vector and desired output, in Boltzmann Machine only the input vector is provided.

- We want to maximize the product of the probabilities the Boltzmann Machine assigns to the binary vectors in training set.
- It is equivalent to maximizing the probability that we would obtain exactly the N training cases.

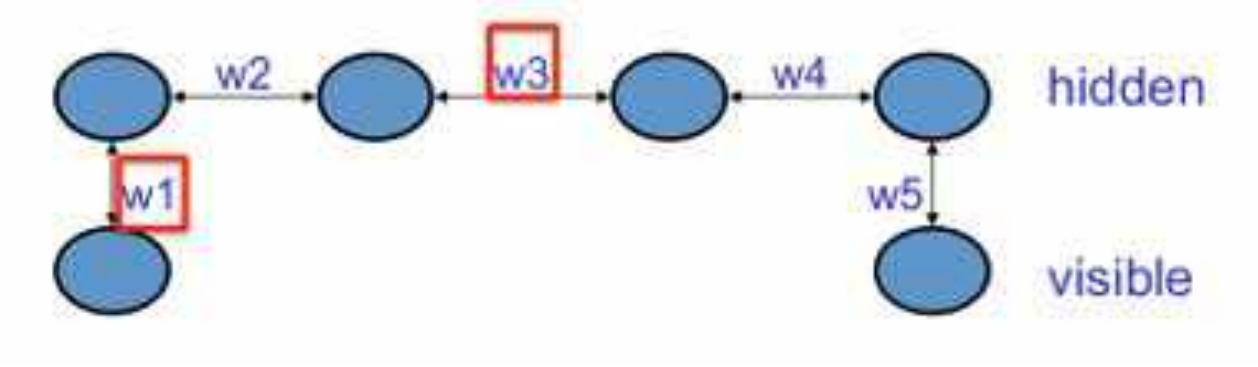
Assumptions

The goal of Boltzmann learning is to produce a NN that categorize input patterns according to Boltzmann distribution. Two assumptions are made:

1. Each environmental vector persists long enough for the network to reach thermal equilibrium.
2. There is no structure in the sequence in which environmental vectors are clamped to the visible units of the network.

Why Learning could be difficult

- Consider a chain of hidden units with visibile units attached at two ends



Training: We want the two visibile units to be in opposite states.

Solution: The product of all the weights must be negative. If all are positive, then turning on one unit will turn on the next unit and eventually the two visibile units will be in same state.

Why Learning could be difficult

Difficulty: To modify w1 and w5, we need to know w3 (and the weights of other hidden units too).

Because, if w3 is negative, then we need to modify w1 in a different way than what we would do if w3 is positive.

So to change one weight in a right direction, we need to know all the other weights.

Boltzmann Learning Algorithm

The learning procedure mainly divided into three phases:

1. Clamping phase
2. Free-running phase
3. Learning phase

Algorithm

1. **Initialization**: Set weights to random numbers in [-1, 1]
2. **Clamping Phase**: Present the net with the mapping it is supposed to learn by clamping input and output units to patterns. For each pattern perform simulated annealing on the hidden units in the sequence $T_0, T_1, ..., T_{final}$. At the final temperature, collect statistics to estimate the correlations

$$\rho_{ji}^{+} = \langle s_j s_i \rangle^{+} \qquad (j \neq i)$$

here $\langle s_j s_i \rangle^{+} = \sum_{\mathbf{s}_\alpha \in \Im} \sum_{\mathbf{s}_\beta} P(\mathbf{S}_\beta = \mathbf{s}_\beta | \mathbf{S}_\alpha = \mathbf{s}_\alpha) s_j s_i$

where $\mathbf{s}_\alpha$ and $\mathbf{s}_\beta$ represents the vector of visible and hidden neurons respectively

Algorithm

3 **Free-running Phase**:Repeat calculations in step 2, but this time only clamp the input units. Hence, at the final temperature, estimate the correlations

$$\rho_{ji}^{-} = \langle s_j s_i \rangle^{-} \qquad (j \neq i)$$

here $\langle s_j s_i \rangle^{-} = \sum_{\mathbf{s}_\alpha \in \Im} \sum_{\mathbf{s}_\beta} P(\mathbf{S}_\beta = \mathbf{s}_\beta) s_j s_i$

4 **Learning Phase**: updating the weights using the learning rule

$$\triangle w_{ji} = \eta(\rho_{ji}^{+} - \rho_{ji}^{-})$$

η is **learning parameter** depending upon T ($\eta = \frac{\epsilon}{T}$)

5 **Iteration**: Iterate steps 2 to 4 until the learning procedure converges with no more changes with synaptic weight $w_{ji} \ \forall \ j, i$.